

Development of a Reference Application for the OWASP Mobile Application Security Testing Guide (MASTG)

BACHELOR'S THESIS
Dominic Kronig

Universität Bern
Bern, November 26, 2025



First Reviewer: **Prof. Dr. Vorname Name**
 Universität XY
 Institut XY

Second Reviewer: **Vorname Name**
 Universität XY
 Institut XY

Abstract: The OWASP Mobile Application Security project is the industry standard when it comes to android mobile application security. It comprises all aspects of mobile application security, from documenting weaknesses to providing verification standards. Part of it is the Mobile Application Security Testing Guide (MASTG), which gives detailed instructions on how to thoroughly test mobile applications. Complementing the MASTG are so called "reference apps", which serve as a source of learning in the form of including intentionally built in weaknesses, which can be found and exploited using the MASTG, as to teaching mobile developers what to avoid in their own applications. This BA thesis aims to evaluate the state of the art of said reference apps, conduct a literature review on the relevancy and usage of the MASTG and related OWASP fields in the academic world, as well as develop a new MASTG reference app. This reference app includes both dynamic and static vulnerabilities and documents each vulnerability thoroughly, including where it can be found in the code, how to exploit it, and how to fix it.

Contents

1	Introduction	1
1.1	Section	2
1.1.1	SubSection	2
2	Related Work	3
2.1	State-Of-The-Art in OWASP MASTG Reference Apps	3
2.1.1	Methodology	3
2.1.2	Findings	4
2.2	Literature Review - Academic relevance of OWASP MASTG	7
2.2.1	Methodology	8
2.2.2	Findings	8
	List of Figures	13
	Bibliography	18

1 Introduction

Start writing here 1...

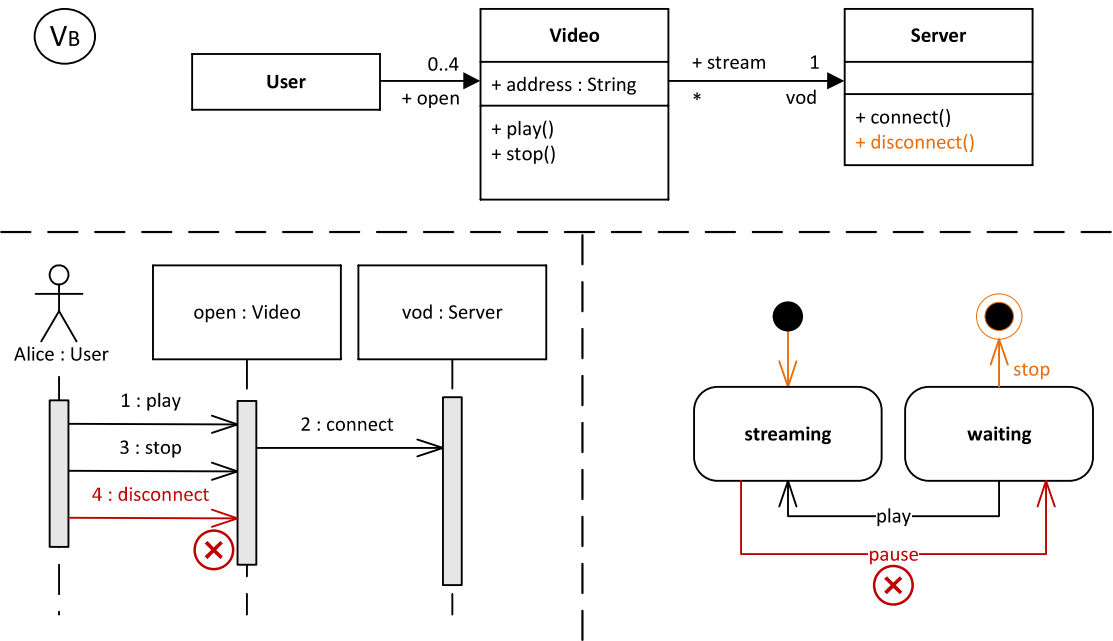


Figure 1.1: Figure description here

Figure 2.3

- one
 - two
1. one
 2. two

In Definition 1.1...

Definition 1.1

$$e = mc^2$$

Lemma 1.0.1

$$e = mc^2$$

Proof. $e = mc^2$

□

Requirement 1.0.1

$$e = mc^2$$

Example 1.0.1

$$e = mc^2$$

```
1 // Hello.java
2 import java.awt.Graphics;
3 public class Hello extends JApplet {}
```

Algorithmus 1.1: foo

```
1 function helloWorld(in name) {}
```

Algorithmus 1.2: foo

SOME NOTE

(TODO: HERE A TODO!)

1.1 Section

1.1.1 SubSection

SubSubSection

Pragraph

SubPragraph

2 Related Work

There are two types of related work for this BA thesis. The first consists of already existing OWASP MASTG reference apps for android. The second is about academic works making use of the MASTG, an industry standard, and its reference apps.

2.1 State-Of-The-Art in OWASP MASTG Reference Apps

At the time of writing, there are currently 19 MASTG reference apps listed on the OWASP page and its corresponding github repository [1]. To evaluate the state of the art of these apps, it is necessary to examine each one of them in order to identify and categorize their implemented weaknesses. Only 8 of said apps had both available sourcecode as well as some form of documentation of their weaknesses. These were then mapped by the author to their corresponding OWASP MASWE counterparts.

2.1.1 Methodology

Each of the 19 reference apps has been evaluated as to whether they have any source code available at all, and if so, whether they have any documentation of their weaknesses. Both of these criteria are necessary for them to be properly evaluated within the framework of this BA thesis. There are four "Android Uncrackable" reference apps, named L1 [2], L2 [3], L3 [4] and L4 [5]. All four of these do not have any documentation on their implemented weaknesses, and are hence not included in this review. The "Android License Validator" [6] by the same author does also not include any such documentation, and is excluded on the same grounds as well. The other apps excluded for the same reason are: DVHMA [7], Digitalbank [8], DodoVulnerableBank [9], disable-flutter-tls-verification [10], and MASTTestApp-Android-NETWORK [11]. The MASTG-Hacking-Playground-Kotlin-App [12] has been excluded from this literature review, since there exists an identical Java version of this app, which has been evaluated.

The following apps have been included in the review: AndroGoat [13], DIVA Android [14], InsecureBankv2 [15], MASTG-Hacking-Playground [16], OVAA [17], InsecureShop [18], Finstergram [19] and Android BugBazaar [20].

2.1.2 Findings

Across the 8 evaluated OWASP MASTG reference apps, 162 weaknesses were mapped to their respective MASWE counterpart and grouped by their vulnerability category (Storage, Crypto, Auth, etc.).

Lack of vulnerability documentation

A reoccurring issue with many of the reference apps is their documentation of the implemented vulnerabilities. 11 of the listed reference apps do not have any documentation of their weaknesses at all, which led to their exclusion from this evaluation. Those apps that do have some form of documentation are often incomplete, and almost always only consist of a single list, naming the implemented weaknesses and leaving it at that. None of the apps map the implemented weaknesses to their OWASP MASWE counterparts, despite them being a MASTG reference app, designed for this very purpose. Only one app ([TODO: add source and specific name here, finstergram?](#)) actually documents what the weaknesses entail, how they are implemented, and how to fix or exploit them.

Since these apps are meant by OWASP to be educational, it does not suffice to only implement the weaknesses and leave as is. The app developed in this BA thesis aims to fill this hole by not only documenting all implemented weaknesses, but adding documentation for each specific weakness, consisting of where it is found, what lines of code are of relevancy, how to exploit them, as well as how to fix them.

Uneven distribution of vulnerability coverage

At the time of writing, there are a total of 117 MASWE vulnerabilities across 8 categories documented by OWASP. Of these 117 vulnerabilities, only 38 are actually covered by any of these reference apps.

([TODO: CREATE GRAPH DISPLAYING VULNERABILITY COVERAGE GROUPED BY APP, INSERT HERE](#)) Since there are 162 weaknesses implemented, but only 38 different types of weaknesses as used by the MASTG, this leaves a total of 79 vulnerabilities, as documented and classified by OWASP MASWE and referenced in the MASTG, completely without representation. This further means that there are some vulnerabilities which are heavily favoured to be implemented in these reference apps, with them being added in high frequency, while many more vulnerabilities see no coverage at all.

To make this point more clear; the ten most frequently covered vulnerabilities are in total implemented 75 times, meaning that they represent 46% (75/162) of all implemented vulnerabilities, despite only covering 8.5% (10/117) vulnerabilities as referenced

by MASTG. The most frequently implemented vulnerability, as seen in Figure 1, is MASWE-0066, which deals with insecure intents. Of the ten most frequently covered vulnerabilities, 5 are of the category "Platform".

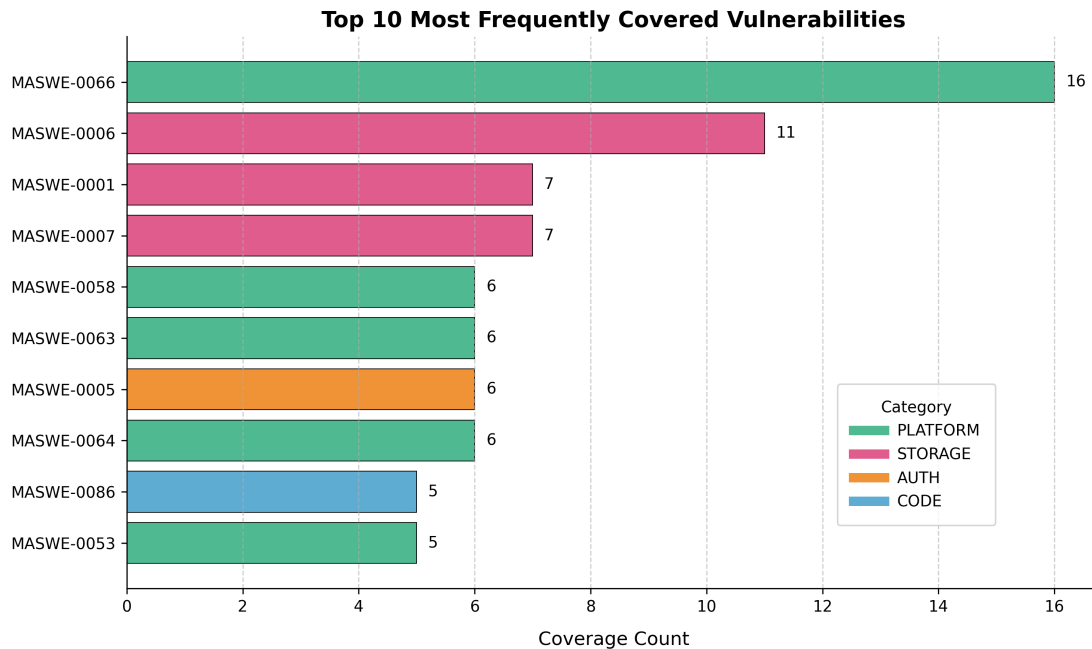


Figure 2.1: Barplot depicting the ten most frequently covered types of OWASP MASWE vulnerabilities, colored according to their respective category.

Coverage of vulnerability categories

There is not only a favouring of certain specific MASWE vulnerabilities, but also an uneven coverage of the vulnerability categories themselves as well. When viewed percentage-wise, as seen in figure 2.2., some categories see a high percentage of their vulnerabilities covered, such as Platform with 72.7% or Storage with 66.7%, while others see very little to no coverage at all, such as both Crypto and Auth with only 10% and worst of all Privacy with a vulnerability coverage of 0%.

Viewing the coverage in absolute rather than percentage values makes it look a lot more un-even. As seen in Figure 2.3, all categories have 2 to 6 types of vulnerability types covered, with the only outliers being the category Platform, which has a total of 16 of 22 vulnerabilities implemented, and Privacy, which has no vulnerabilities at all covered. This shows that vulnerabilities are covered rather evenly across the 8 categories,

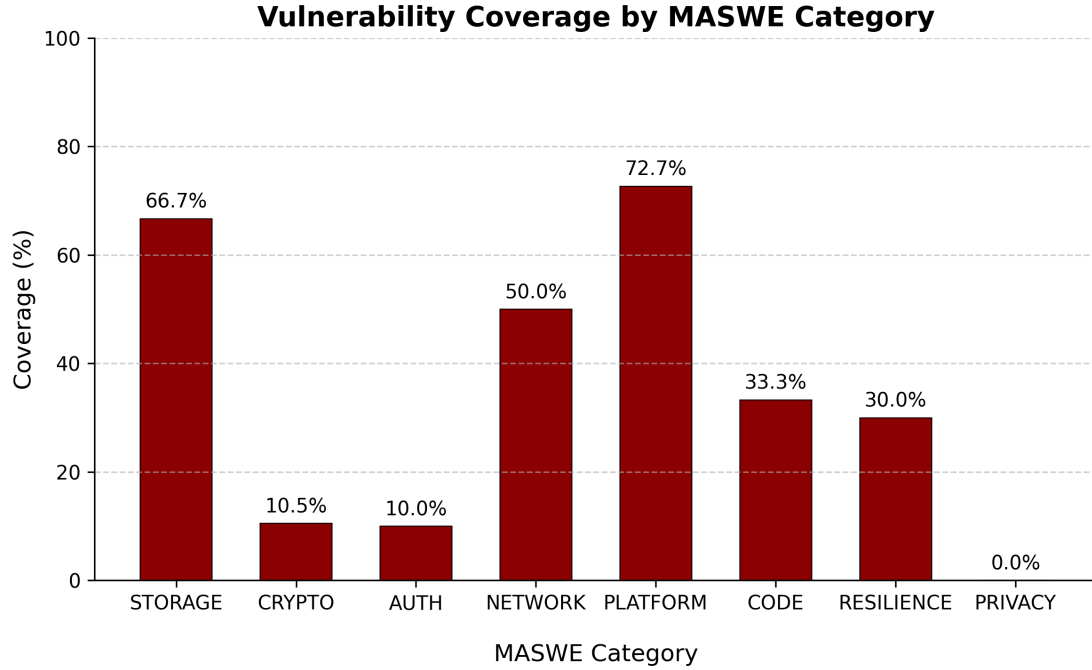


Figure 2.2: (TODO: ADD IMAGE CAPTION)

regardless of the amount of vulnerabilities each category has, which explains the large differences in percentage-wise coverage as seen in the previous Figure 2.2.

The remarkably high coverage of Platform vulnerabilities further enforces the findings of Figure 2.1, meaning that not only are 5 of the 10 most frequently covered vulnerabilities of type Platform, the category itself is also the most broadly implemented category when compared to others. The complete lack of coverage of Privacy vulnerabilities can in part be accounted for by the fact that the category itself is newer than the other 7, only having been introduced in MASVS version 2.1.0. (TODO: ADD SOURCE HERE). Also, the vulnerabilities as described in Privacy are partly more abstract in nature, not necessarily being implementable in the code itself, such as "MASWE-0111: Inadequate Privacy Policy".

Nevertheless, it can be concluded that the categories are un-evenly covered, with Platform receiving a great deal more of attention than the other categories, and Privacy receiving none. Also, it is clear that the coverage of many categories in these state-of-the-art reference apps is lacking, with only a concerning 10% of all Crypto and Auth vulnerabilities, as documented by MASWE and tested by MASTG, actually being covered. This despite both of these categories being of great importance, and making up a

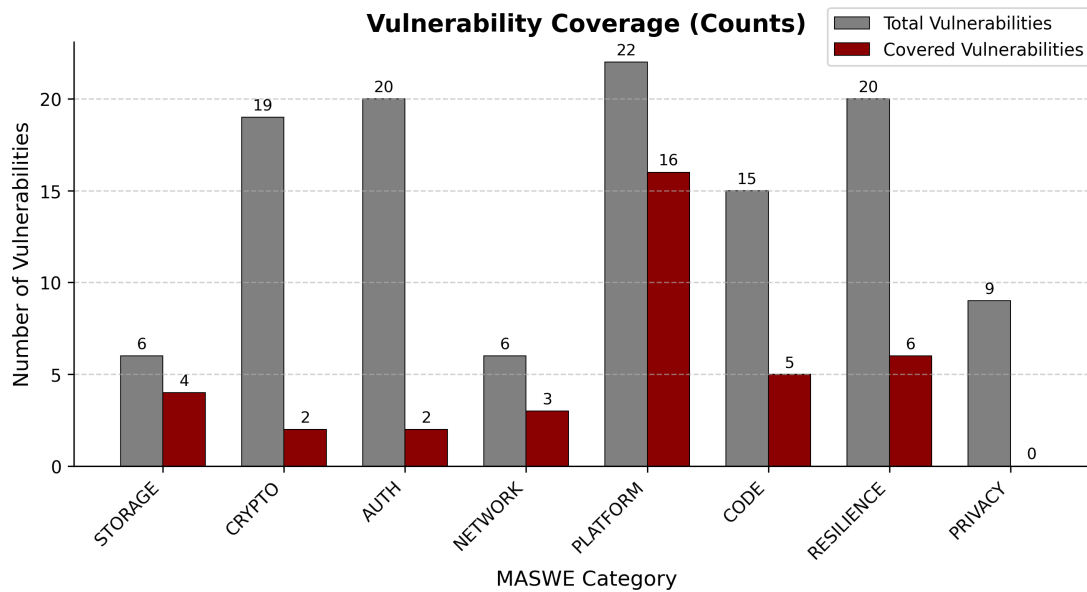


Figure 2.3: (TODO: ADD IMAGE CAPTION)

large part of what mobile security as conducted by OWASP deals with, considering they make up a third (33%) of all vulnerabilities as documented by MASWE and tested by MASTG.

The reference app developed in this BA thesis aims to solve this issue, by covering the vulnerabilities more broadly, and shifting the focus from over-favoured Categories such as Platform to more neglected ones, such as Crypto and Auth.

2.2 Literature Review - Academic relevance of OWASP MASTG

The OWASP MASTG is the industry standard for mobile app security. But what relevancy does it have in the academic world, and what exactly is it used for? To answer this question, a literature review was conducted to evaluate papers on their use of the OWASP MASTG, as well as related OWASP parts, e.g., the MASTG reference apps, the MASWE, and the MASVS.

2.2.1 Methodology

This literature review was conducted by searching for relevant papers that include and deal with the keywords "OWASP" and "MASTG" on google scholar. Only papers written in english have been considered. Furthermore, all evaluated papers have been written in the last two years, with no paper having an older publishing year than 2023. The top 22 papers matching these criteria have been carefully read, with seven of them discarded because their research focus did not match the OWASP MASTG. This left 15 papers, which have then been evaluated for their usage of OWASP MASTG and grouped by similarities.

2.2.2 Findings

Of the 15 papers evaluated, 7 used OWASP MASTG to evaluate real-world android apps based on whether they adhere to the security standard set by OWASP MASVS. 6 papers mapped android app security vulnerabilities to the OWASP MASVS. Three papers made use of the 5-steps penetration testing guide as layed out by OWASP MASTG, in order to conduct penetration tests on android apps. Three papers also used the MASTG reference apps by evaluating vulnerability assessment tools on them in order to verify their accuracy in finding such vulnerabilities. Some (TODO: replace "Some" with actual count) of these papers then compared their findings to results found when conducting the same tests on real-world apps. Lastly, four papers mentioned the OWASP MASTG in discussion, either by the authors, comparing the OWASP mobile project to other Mobile Security Frameworks, or by interviewees as a tool used.

Cryptographic security shortcomings in african mobile financial applications

One paper [21] used the OWASP MASTG & MASVS to evaluate 18 android apps belonging to different african financial institutions based on their mobile security. They chose MASVS as a testing framework, since it is specifically designed to "be a guide and checklist for evaluating the security of mobile applications". They applied the strictest level of MASVS, L2, and focused solely on unauthenticated client-side testing, following the black-box testing approach. They did not implement all MASVS tests. Nevertheless, their findings in terms of security shortcomings are quite damning. Some of the worst findings are as follows. All 18 apps allowed for screenshotting of sensitive data. 14 of the apps used outdated and thus insecure cryptographic protocols such as MD5, SHA-1 and DES. 10 apps used security libraries that are outdated and known to have

vulnerabilities. Furthermore, eight apps were using insecure random number generation for their cryptographic protocols, and 5 apps have no certificate pinning. This shows a clear and dramatic shortcoming in terms of cryptographic mobile security. All of the above mentioned shortcomings are well documented in the OWASP MASWE, with examples for how to test for such weaknesses described in the MASTG and are part of the verification standard MASVS. This shows that the OWASP mobile security project has a great use case in the real world, and is far from being outdated or redundant. In a conducted survey by the authors of the study, 78% of respondents said they avoided implementing security due to a lack of expertise and knowledge about security. By creating a intentionally vulnerable reference app which greatly covers vulnerabilities in the cryptographic sector, this BA thesis fills a hole in the current landscape of available reference apps, which do not cover the CRYPTO section of the MASWE to a sufficient extent. It builds on the issues found and discussed in this paper, by enabling mobile developers to deepen their expertise and knowledge of cryptographic vulnerabilities in a hands-on way.

Lack of basis for Mutation Testing for android app security testing

Another paper [22] uses Mutation Testing for android app security testing. To do so, the authors used the OWASP Top 10 Mobile Risks to assess common android app security vulnerabilities, and create mutation operators based on these vulnerabilities. They then perform security tests against the mutants generated by the mutation operators, in the hope that these mutants are detected as vulnerabilities. They do this on 10 android reference apps from the MASTG reference app catalog. They find that four of the eight desired generated mutant operators are not represented among the mutants at all, while two other are heavily overrepresented. To be more specific, all three tapjacking vulnerability mutant operators and the "ImplicitPendingIntent" operator have no representation in the apps, whereas ImproperExport and HardcodedSecret are both heavily overrepresented. This finding further enforces my own findings upon analyzing the implemented vulnerabilities in the reference apps, and can be thus explained. Of the 162 vulnerabilities implemented by these apps which have been gathered and categorized as seen in Section 2 of this Thesis (TODO: probably add proper reference here, not sure how yet), only 1 deals with Tapjacking attacks, the same holds for implicit pending intents. On the contrary, eight vulnerabilities deal with hardcoded secrets. Because of uneven distribution of implemented vulnerabilities, some mutants are more easily generated than others. By covering more of the vulnerabilities in this BA thesis' reference app that have

this far been neglected by other reference apps, this BA thesis further enables the study of mutation testing for android app security, as done in this paper.

Security shortcomings in the american mHealth sector

Another paper [23] used the OWASP Mobile Audit to evaluate 95 apks of real-world apps which are part of the mobile health sector in the USA. The OWASP Mobile Audit is an OWASP project which coexists with other OWASP projects such as the MASTG and MASWE, serving as a tool for static code security analysis of android apps. It "applies static application security testing (SAST) aligned with the MASTG to identify vulnerabilities, insecure configurations, certificate handling issues, and potential malware signatures." The findings were once again quite devastating: these apps dealing with sensitive user health data had on average 44 critical, 2'194 high, 3'728 medium, 1'735 low, and 1'469 vulnerabilities with no severity assigned. Frequently found vulnerabilities include insecure storage of authentication tokens, absence of certificate pinning, 42 of the 95 apps making use of unencrypted transmission of data containing personal health information over http and reliance on outdated cryptographic primitives. 49.3% of applications still rely on RSA SHA-1 signatures, exposing them to collision-based exploits such as Janus, which permits undetected APK modification and repackaging. 22 apps were found to be configured to trust all TLS certificates. Furthermore, there 2'252 instances of unnecessarily exported broadcast receivers and 1'232 cases of exported permissions without defined protection levels found. This paper once again underlines the huge shortcoming of apps in the cryptographic sector of mobile security, even when it comes to sensitive user data. All of the found vulnerabilities are once again well documented by the OWASP mobile security project, and need't exist anymore. The authors themselves conclude: "Weak enforcement is compounded by insecure development practices. Developers should adopt the OWASP MASVS, embed security linters into IDEs, [...]" [23], emphasizing the real-world importance of the OWASP MASVS, the MASTG and the related reference apps. The reference app developed in the course of this BA thesis works to solve the issues found by the authors of this paper, by including many vulnerabilities in the cryptographic sector, since many of the found vulnerabilities deal with cryptographic issues, while all of the existing reference apps are lacking when it comes to encompassing a thorough implementation of cryptographic vulnerabilities.

Security Assessment for Digital Wallet Payment Partner Applications using the OWASP Method: A Case Study in Indonesia

This paper [24] uses the OWASP MASTG and MASVS to assess the security standard of digital wallet payment partner mobile app. The authors make use of both dynamic (DAST) and static (SAST) application security testing, map the found vulnerabilities to the MASWE and use the OWASP Risk Rating Methodology to assess the security risks of the found vulnerabilities. They find vulnerabilities with both the static and the dynamic approach, showing that it is important for reference apps to include both dynamic and static vulnerabilities, if they desire to mimic real-world app vulnerabilities. The reference app developed in this BA thesis builds on this discovery by aiming to include both static and dynamic vulnerabilities. (TODO: change this from future to present/past tense, once this has actually been achieved.) The paper finds that of the 84 MASVS verification points, 68 were labelled as "pass", 10 were deemed not applicable, 6 were failed (vulnerability found). Multiple papers and articles, as referenced by the authors, show that the OWASP mobile security project presents a viable solution for security assessment. They mention that financial transaction apps are particularly common to have a high vulnerability rate, and that insecure data storage stands out compared to other vulnerabilities with a vulnerability rating of up to 76%. (TODO: go find out what exactly is meant with this magic term "rating". sounds fishy as written at the moment.) My reference app developed for this BA thesis builds on this, by covering all vulnerabilities related to storage, as described by the OWASP MASWE. By doing so, my app mimics vulnerabilities as they are found in real-world apps.

SoK: Hardening Techniques in the Mobile Ecosystem — Are We There Yet?

The OWASP MASVS advises mobile app developers to implement so called hardening techniques, which serve as a layer of device protection by preventing tampering with the device and data leakage, detecting jailbreaks etc. This paper [25] aims to understand to which extend real-world apps adhere to the MASVS and implement such hardening techniques. To do so, they created HALY, a framework which allows for static as well as dynamic analysis of the adoption of hardening techniques in mobile apps. They discuss how MASVS-RESILIENCE specifies multiple hardening techniques which apps should implement as to conform to modern security standards. They use the MASVS as a base of operation, since "MASVS is also the standard Android apps are tested against in case apps opt for the independent security review reflected in Google Play's data safety section.", [25] which emphasizes and displays the importance of the recommendations

and practices laid out in the standards laid out by the OWASP mobile security project. They build their framework with the goal in mind to be able to detect vulnerabilities in three out of the four categories defined by MASVS-RESILIENCE, namely category 1 (integrity validation of the platform), 3 (anti-static analysis mechanisms) and 4 (anti-dynamic analysis mechanisms). The authors validate the accuracy of their framework by testing it on seven open source apps, under them being four Android and two iOS OWASP reference apps. They then go on to use their framework to evaluate 2646 apps, both available on Android and iOS, for their use of hardening techniques. Their findings show that iOS apps heavily under-perform when it comes to implementing hardening techniques, as opposed to apps on Android devices. Of the 2646 apps evaluated, 24.1% of Android and 73.6% of iOS apps implement less than half of the protection techniques recommended by MASVS-RESILIENCE. Only 26 Android apps adopt all eight, while only one iOS app adopts all seven. This paper not only shows the real-world importance of OWASP MASTG and MASVS, by mentioning its role as the standard used in Google Play's data safety section, it displays the merit of MASTG reference apps and MASVS to the academic world, as a basis on which to conduct important research. The research furthermore shows that there are still many mobile apps which do not follow the best security practices defined by OWASP. The OWASP MASTG reference app developed as a part of this BA thesis not only provides further basis on which future academic work similar to this paper can build on, but also serves as an example from which mobile developers can learn from to improve the security standards of mobile apps used globally.

List of Figures

1.1	Figure description here	1
2.1	Barplot depicting the ten most frequently covered types of OWASP MASWE vulnerabilities, colored according to their respective category.	5
2.2	(TODO: ADD IMAGE CAPTION)	6
2.3	(TODO: ADD IMAGE CAPTION)	7

Bibliography

- [1] “Owasp mastg reference applications overview,” 09 2025, as of commit Nr: ffde0bdd9bdfd375d93ddb433bd3134e3977c64d. [Online]. Available: <https://github.com/OWASP/mastg/tree/ffde0bdd9bdfd375d93ddb433bd3134e3977c64d/apps/android>
- [2] B. Mueller, “Android uncrackable 11,” 09 2025, as of commit Nr: ffde0bdd9bdfd375d93ddb433bd3134e3977c64d. [Online]. Available: <https://github.com/OWASP/mastg/blob/ffde0bdd9bdfd375d93ddb433bd3134e3977c64d/apps/android/MASTG-APP-0003.md>
- [3] —, “Android uncrackable 12,” 09 2025, as of commit Nr: ffde0bdd9bdfd375d93ddb433bd3134e3977c64d. [Online]. Available: <https://github.com/OWASP/mastg/blob/ffde0bdd9bdfd375d93ddb433bd3134e3977c64d/apps/android/MASTG-APP-0004.md>
- [4] —, “Android uncrackable 13,” 09 2025, as of commit Nr: ffde0bdd9bdfd375d93ddb433bd3134e3977c64d. [Online]. Available: <https://github.com/OWASP/mastg/blob/ffde0bdd9bdfd375d93ddb433bd3134e3977c64d/apps/android/MASTG-APP-0005.md>
- [5] —, “Android uncrackable 14,” 09 2025, as of commit Nr: ffde0bdd9bdfd375d93ddb433bd3134e3977c64d. [Online]. Available: <https://github.com/OWASP/mastg/blob/ffde0bdd9bdfd375d93ddb433bd3134e3977c64d/apps/android/MASTG-APP-0015.md>
- [6] —, “Android license validator,” 09 2025, as of commit Nr: ffde0bdd9bdfd375d93ddb433bd3134e3977c64d. [Online]. Available: <https://github.com/OWASP/mastg/blob/ffde0bdd9bdfd375d93ddb433bd3134e3977c64d/apps/android/MASTG-APP-0002.md>

- [7] M. H. Achim D. Brucker, “Dvhma,” 08 2018, as of commit Nr: 30f76ee118f4cc37bdd9dc897f5241d872bbe2b5. [Online]. Available: <https://github.com/logicalhacking/DVHMA/tree/30f76ee118f4cc37bdd9dc897f5241d872bbe2b5>
- [8] K. S. Abhinav Sejjal, “Digitalbank,” 08 2015, as of commit Nr: 9fae450849d221a22cb16d6491ecabad4c97fbd9. [Online]. Available: <https://github.com/CyberScions/Digitalbank/tree/9fae450849d221a22cb16d6491ecabad4c97fbd9>
- [9] “Dodovulnerablebank,” 10 2015, as of commit Nr: 8b9ac17fe4619720650169b86f9bb842aca5976c. [Online]. Available: <https://github.com/CSPF-Founder/DodoVulnerableBank/tree/8b9ac17fe4619720650169b86f9bb842aca5976c>
- [10] A. B. Jeroen Beckers, “disable-flutter-tls-verification,” 05 2025, as of commit Nr: 9dec9e11afa3269095a130de1e270a1f094c08f. [Online]. Available: <https://github.com/NVISOsecurity/disable-flutter-tls-verification/tree/9dec9e11afa3269095a130de1e270a1f094c08f>
- [11] J. Sydseter, “Mastestapp-android-network,” 05 2025, as of commit Nr: 5e5278c82f77ffc4cd5db9efc32d0b98dc0d7915. [Online]. Available: <https://github.com/sydseter/MASTestApp-Android-NETWORK/tree/5e5278c82f77ffc4cd5db9efc32d0b98dc0d7915>
- [12] S. Schleier, “Mastg-android-kotlin-app,” 10 2022, as of commit Nr: 211e1cb27be30c575a36c6ff3bbe6dfb43182f9f. [Online]. Available: <https://github.com/OWASP/MASTG-Hacking-Playground/tree/211e1cb27be30c575a36c6ff3bbe6dfb43182f9f>
- [13] S. Patnayak, “Androgoat,” 04 2020, as of commit Nr: 1f383989bf8435ba8aea245439e1b7f68f2f985d. [Online]. Available: <https://github.com/satishpatnayak/AndroGoat/tree/1f383989bf8435ba8aea245439e1b7f68f2f985d>
- [14] A. Jakhar, “Diva android,” 01 2016, as of commit Nr: 2bb63b531ba66c22563ac0911973389e2a0cb723. [Online]. Available: <https://github.com/payatu/diva-android/tree/2bb63b531ba66c22563ac0911973389e2a0cb723>
- [15] A. S. Dinesh Shetty, “Insecurebankv2,” 11 2019, as of commit Nr: 3c8cc3a3ce45ed56d0f58f236b541b41f992e51f. [Online]. Available: <https://github.com/dineshshetty/Android-InsecureBankv2/tree/3c8cc3a3ce45ed56d0f58f236b541b41f992e51f>

-
- [16] S. Schleier, “Mastg-hacking-playground,” 10 2022, as of commit Nr: db219a1011e6735cf0c6c08ba929a27ef40e1873. [Online]. Available: <https://github.com/OWASP/MASTG-Hacking-Playground/tree/db219a1011e6735cf0c6c08ba929a27ef40e1873>
- [17] “Ovaa,” 7 2024, as of commit Nr: 4c60d202918e33d3dc9dd7574260f75f761f6e2b. [Online]. Available: <https://github.com/oversecured/ovaa/tree/4c60d202918e33d3dc9dd7574260f75f761f6e2b>
- [18] R. G. Gaurang Bhatnagar, “Insecureshop,” 09 2023, as of commit Nr: 76ed1772a12c61a60f4a685770482e0e1f6cf4a7. [Online]. Available: <https://github.com/hax0rgb/InsecureShop/tree/76ed1772a12c61a60f4a685770482e0e1f6cf4a7>
- [19] F. W. Jonas Mayer, “Finstergram,” 07 2024, as of commit Nr: 11fd8b7e24dada74a751ca112956bb7944a17ca5. [Online]. Available: <https://github.com/netlight/finstergram/tree/11fd8b7e24dada74a751ca112956bb7944a17ca5>
- [20] A. K. Amit Kumar Prajapat, Vedant Wayal, “Android bugbazaar,” 04 2024, as of commit Nr: fef6a899a684245eb70b2ab4be926f62be849ed9. [Online]. Available: <https://github.com/payatu/BugBazaar/tree/fef6a899a684245eb70b2ab4be926f62be849ed9>
- [21] T. H. Chiboora, L. Chacha, T. Byagutangaza, and A. Gueye, “Evaluating mobile banking application security posture using the owasp’s masvs framework,” in *Proceedings of the 6th ACM SIGCAS/SIGCHI Conference on Computing and Sustainable Societies*, ser. COMPASS ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 99–106. [Online]. Available: <https://doi.org/10.1145/3588001.3609367>
- [22] E. Vasconcelos, M. Delamaro, and S. Souza, “Mutation testing to support the security testing of android applications,” in *Anais do IX Simpósio Brasileiro de Testes de Software Sistemático e Automatizado*. Porto Alegre, RS, Brasil: SBC, 2024, pp. 29–38. [Online]. Available: <https://sol.sbc.org.br/index.php/sast/article/view/30213>
- [23] L. Stevenson and S. Das, “"your doctor is spying on you": An analysis of data practices in mobile healthcare applications,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.06015>

- [24] M. Saifulhakim and A. Fajar, “Security assessment for digital wallet payment partner applications using the owasp method: A case study in indonesia,” *Journal of System and Management Sciences*, vol. 14, pp. 337–357, 02 2024.
- [25] M. Steinböck, J. Troost, W. Van Beijnum, J. Seredynski, H. Bos, M. Lindorfer, and A. Continella, “Sok: Hardening techniques in the mobile ecosystem — are we there yet?” in *2025 IEEE 10th European Symposium on Security and Privacy (EuroSP)*, 2025, pp. 789–806.

Erklärung

gemäss Art. 30 RSL Phil.-nat.18

Name/Vorname:

Matrikelnummer:

Studiengang:

Bachelor ☐

Master ☐

Dissertation ☐

Titel der Arbeit:

LeiterIn der Arbeit:

Ich erkläre hiermit, dass ich diese Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen benutzt habe. Alle Stellen, die wörtlich oder sinngemäss aus Quellen entnommen wurden, habe ich als solche gekennzeichnet. Mir ist bekannt, dass andernfalls der Senat gemäss Artikel 36 Absatz 1 Buchstabe r des Gesetzes vom 5. September 1996 über die Universität zum Entzug des auf Grund dieser Arbeit verliehenen Titels berechtigt ist.

Für die Zwecke der Begutachtung und der Überprüfung der Einhaltung der Selbständigkeitserklärung bzw. der Reglemente betreffend Plagiate erteile ich der Universität Bern das Recht, die dazu erforderlichen Personendaten zu bearbeiten und Nutzungshandlungen vorzunehmen, insbesondere die schriftliche Arbeit zu vervielfältigen und dauerhaft in einer Datenbank zu speichern sowie diese zur Überprüfung von Arbeiten Dritter zu verwenden oder hierzu zur Verfügung zu stellen.

Ort/Datum

Unterschrift

